This instructs Twig to load `index.html.twig` template – which will be delegated to the `FilesystemLoader` we instantiated with the search path `"./views"` – read in the contents and compile. The second argument to `::render()` is an associative array. The keys of this array become the variable names made available to the template when it compiles. This means that the `{{ title }}` variable should be replaced with a `RENDERED WITH TWIG` string when our template compiles.

With a `cli-server` container launched from the directory, you placed the `index.php` and `views/index.html.twig` files (and with Twig installed into `vendor/twig`, composer autoloader in `vendor/autoload.php`, etc.), make the request to `http://localhost:8080/index.php`, the result should be:

```
$ curl http://localhost:8080/index.php
...
  <head>
    <title>RENDERED WITH TWIG</title>
  </head>
  <body>
    <p>PLACEHOLDER CONTENT</p>
  </body>
...
```

Great success! You can see that we have the correct `title` metadata and that our template rendered the default `body` content because we didn't set a `content` variable when we called the `::render()` method. You can also obtain a template object with the `Twig\Environment::load()` method and call `render()` on that. The example above could also have been written as:

```
$template = $twig->load('my-first.html.twig');
print $template->render([
    'title' => 'RENDERED WITH TWIG'
]);
```

Twig offers so much more than the simple variable interpolation and control structures demonstrated here, but the basic grammar of a template won't change beyond those three tags. Your own output might vary slightly on spacing and indenting from the above, but this doesn't impact the structure of the HTML. Slight variations are bound to creep in; a nicely